

# BÁO CÁO THIẾT KẾ GIẢI PHÁP

## SS14 HW04 - Sự đánh đổi giữa tự do và tập trung

Sinh viên: Trương Hà Cẩm Linh | Lớp: IT214 | Mã sinh viên: PTIT056

### 1. Phân tích yêu cầu tích hợp Voucher

Voucher không chỉ là phép trừ tiền đơn giản. Luồng phải kiểm tra mã tồn tại, thời hạn, điều kiện giá trị đơn tối thiểu, phạm vi sản phẩm, số lượt còn lại và quyền sử dụng của khách hàng. Sau đó hệ thống giữ lượt dùng để tránh hai đơn cùng tiêu thụ một voucher, tính mức giảm và cập nhật số tiền Payment phải thu.

- Nhận input: customerId, voucherCode, tổng tiền ban đầu và thông tin đơn.
- Voucher Service xác thực và giữ lượt dùng theo orderId.
- Tính discount và finalAmount; Order cập nhật tổng tiền mới.
- Payment chỉ trừ finalAmount sau khi cập nhật thành công.
- Nếu bước sau lỗi: hoàn tiền nếu cần, giải phóng voucher và hủy đơn.

### 2. Giải pháp Choreography

Voucher Service lắng nghe OrderCreated, kiểm tra voucher rồi phát VoucherApplied. Order Service nhận event để cập nhật tổng tiền và phát event tiếp theo cho Payment. Mỗi service tự quyết định phản ứng, không có bộ điều phối trung tâm.

OrderCreated → VoucherApplied → OrderAmountUpdated → PaymentSuccess → OrderCompleted

#### Ưu điểm

- Các service độc lập, scale consumer riêng và ít phụ thuộc trực tiếp.
- Luồng bất đồng bộ chịu tải tốt và service tạm dừng có thể đọc bù event.

#### Hạn chế

- Thêm Voucher làm tăng topic, event contract và quan hệ ngầm.
- Khó biết đơn đang dừng ở bước nào nếu thiếu tracing tập trung.
- Logic bù trừ bị phân tán, debug event trùng hoặc sai thứ tự khó hơn.

### 3. Giải pháp Orchestration

Checkout Orchestrator quản lý thứ tự gọi Order, Voucher và Payment. Nó lưu ngữ cảnh của Saga để biết bước nào đã hoàn tất và gọi hành động bù trừ tương ứng khi có lỗi.

Tạo Order PENDING → Giữ Voucher → Cập nhật finalAmount → Thanh toán → Order COMPLETED

#### Luồng bù trừ

- Voucher sai: hủy Order.
- Payment lỗi: giải phóng Voucher, sau đó hủy Order.
- Xác nhận Order lỗi sau Payment: hoàn tiền, giải phóng Voucher, hủy Order.

### 4. Bảng so sánh

| Tiêu chí     | Choreography                                   | Orchestration                                      |
|--------------|------------------------------------------------|----------------------------------------------------|
| Thêm service | Ban đầu dễ; về sau tăng event và quan hệ ngầm. | Thêm bước rõ trong workflow; orchestrator lớn dần. |
| Giám sát     | Phải ghép log và trace từ nhiều topic.         | Dễ xem trạng thái và lỗi tại một nơi.              |
| Độ trễ       | Bất đồng bộ, không chờ tuần tự.                | Thường cao hơn do nhiều lời gọi tuần tự.           |
| Mở rộng      | Service và consumer scale độc lập.             | Phải scale orchestrator và tránh nút nghẽn.        |
| Bảo trì      | Event graph phức tạp khi hệ thống lớn.         | Luồng dễ đọc nhưng phụ thuộc bộ điều phối.         |
| Bù trừ       | Phân tán giữa các service.                     | Chính sách compensate tập trung, dễ kiểm soát.     |

### 5. Quyết định kiến trúc

Bài chọn Orchestration. Lý do chính là luồng hiện đã có Voucher, Loyalty Points và Notification; nếu tiếp tục thêm event tự do, chi phí quan sát và debug sẽ tăng nhanh. Orchestrator biến quy trình thành một workflow đọc được, có điểm quan sát chung và chính sách bù trừ rõ ràng.

Đánh đổi được chấp nhận là orchestrator có thêm trách nhiệm và có thể trở thành nút nghẽn. Cách giảm rủi ro là giữ nghiệp vụ cốt lõi trong từng service, để orchestrator chỉ điều phối; đồng thời triển khai nhiều instance và lưu trạng thái Saga bền vững.

### 6. Thiết kế source demo

- checkout-orchestrator (8080): điều phối và compensate.
- order-service (8081): tạo, cập nhật tiền, hoàn tất hoặc hủy đơn.
- voucher-service (8082): kiểm tra SAVE10, giữ và giải phóng voucher.
- payment-service (8083): thanh toán và hoàn tiền.

Demo dùng WebClient và Mono để nối các bước. SagaContext ghi nhận Order, Voucher và Payment đã hoàn tất tới đâu. onErrorResume gọi compensate theo thứ tự ngược, nhờ đó luồng chính và luồng lỗi đều quan sát được tại Orchestrator.

## 7. Kết luận

Choreography phù hợp với ít bước, event ổn định và nhu cầu scale độc lập cao. Khi số service và nhánh bù trừ tăng, Orchestration dễ vận hành và bảo trì hơn. Với StoreX ở giai đoạn hiện tại, bộ điều phối tập trung là lựa chọn hợp lý cho checkout, miễn là không kéo logic nghiệp vụ từ các service vào orchestrator.

## 8. Kịch bản kiểm thử

Kịch bản thành công dùng voucher SAVE10 và số tiền 1.000.000 VND. Voucher giảm 10%, Payment thu 900.000 VND và Order kết thúc ở COMPLETED.

Kịch bản voucher sai dùng mã SAI-MA. Orchestrator nhận lỗi từ Voucher Service và gọi Order Service chuyển đơn sang CANCELLED; Payment chưa được gọi.

Kịch bản thanh toán lỗi dùng số tiền sau giảm vẫn lớn hơn 5.000.000 VND. Orchestrator giải phóng lượt giữ voucher và hủy Order. Nếu lỗi xảy ra sau khi Payment thành công, luồng bù trừ gọi refund trước khi giải phóng voucher.

## 9. Lưu ý khi triển khai production

- Lưu SagaContext vào database thay vì chỉ giữ trong bộ nhớ tiến trình.
- Mọi lệnh pay, refund, apply và release phải có idempotency key theo orderId.
- Áp dụng timeout, retry có giới hạn và Circuit Breaker cho từng lời gọi.
- Gắn correlationId vào log và trace để theo dõi toàn bộ checkout.
- Có cơ chế retry hoặc hàng đợi riêng nếu một hành động compensate thất bại.